How Spotify and Netflix Build Their ML Pipelines

Scaler School of Technology | ML System Design and LLMOps | Week 5 — Session 2 | Post-Lecture Review & Practice

Key Takeaways

Playlists as sentences, songs as words

Spotify's most important insight: you can apply word2vec to playlists and obtain song embeddings that capture cultural co-occurrence — without any explicit genre labels, audio features, or editorial curation. Songs that belong together cluster together because users put them together.

Cold-start is a first-class problem, not an edge case

New songs cannot appear in word2vec embeddings until they accumulate playlist co-occurrences. Spotify's solution is a dedicated audio CNN pipeline: every new song gets an embedding from its audio content alone, on day 1. Cold-start must be designed for explicitly — it cannot be added later.

Netflix optimises for long-term satisfaction, not CTR

Maximising click-through rate produces clickbait. Netflix's north star metric is streaming hours, supplemented by post-play survey scores. The evaluation metric you choose determines what your model learns to optimise. Choose badly and you build a system that harms the product it is supposed to improve.

Freshness strategy is an architectural decision

Spotify updates recommendations weekly. Netflix retrains its ranker hourly. Neither is "right" — the choice depends on business requirements, cost constraints, and how quickly user intent changes. Freshness strategy cannot be bolted on after building the pipeline.

Test Your Understanding

Try answering these without looking at your notes. Answers are on page 3.

Q1. Spotify releases a new song today. Walk through exactly which embedding(s) it will receive and why, covering both the first week and after 30 days.

Q2. Netflix's ranker is retrained hourly but the candidate generation model is retrained daily. Why are these different frequencies? What would break if you retrained both hourly?

Q3. A product manager asks you to add a "trending now" feature to Discover Weekly so new viral songs appear within 24 hours. What change to the Spotify pipeline would be required? What would you give up?

Q4. Your W5S1 design used a two-stage architecture (candidate generation + ranker). Netflix also uses this. But Netflix's candidate model uses two towers while you may have used a simpler approach. Name one advantage and one disadvantage of the two-tower architecture vs. a simpler matrix factorisation baseline.

Design Exercises

Work through these independently or in pairs. These require extending what you learned today, not just recalling it.

Exercise 1: Draw the Spotify Pipeline

Recreate the Spotify Discover Weekly pipeline as a system diagram from memory. Label every box and every arrow. Then annotate each box with: (a) what data it reads, (b) what data it writes, (c) how frequently it runs. Compare your diagram to the case study. Where does yours differ?

Exercise 2: Freshness Sensitivity Analysis

Consider a recommendation system for a news platform (article recommendations). Should you use Spotify's weekly batch approach or Netflix's hourly retraining approach? Write a two-paragraph justification covering: (a) how quickly user intent changes in news vs. entertainment, and (b) what the cost and complexity trade-offs are. Be concrete about what "hourly retraining" actually requires in infrastructure.

Exercise 3: Evaluation Redesign

Netflix switched its north star from CTR to streaming hours, and then added post-play surveys. You are designing the evaluation framework for a short-video platform (think: TikTok). Propose three metrics: one that is easy to game (like CTR), one that captures medium-term satisfaction (like streaming hours), and one that captures long-term health (like 30-day retention). For each, explain how a model could learn to maximise it while actually harming the user experience.

Challenge Problem

The Spotify Podcast Problem

Spotify acquired Gimlet Media and launched podcast recommendations using the same infrastructure that powers music recommendations. Identify three specific reasons why the Discover Weekly pipeline does *not* transfer directly to podcast recommendations. For each reason, propose a concrete engineering change. Consider: episode length and completion signals, content freshness (news podcasts vs. evergreen), and the difference between "one song many times" vs. "one episode once".

Further Reading

- **Spotify Engineering Blog — How Discover Weekly Works** — engineering.atspotify.com, 2015–2017
  The original published explanation of the word2vec on playlists approach. Search: "Spotify Discover Weekly how it works engineering". Read the original post by Chris Johnson and the follow-up by Sophia Ciocca.
- **Netflix Research — Deep Learning for Recommender Systems** — Netflix Tech Blog, available at netflix-techblog.com
  Netflix's published research on the two-tower architecture and multi-objective ranking. Search: "Netflix two-tower recommendation system blog".
- **Covington, Adams, Sargin (2016) — Deep Neural Networks for YouTube Recommendations** — ACM RecSys 2016
  The canonical paper on the two-stage candidate-generation + ranking architecture. YouTube's system is architecturally similar to Netflix's. Free on Google Scholar.
- **Mikolov et al. (2013) — Distributed Representations of Words and Phrases** — NIPS 2013
  The original word2vec paper. Read the abstract and Section 4 (experiments). Understanding why word2vec works helps you understand why Spotify's playlist-as-sentence trick is valid.

Online Resources

- **Netflix Tech Blog** — netflixtechblog.com — detailed engineering posts
- **Spotify Engineering** — engineering.atspotify.com — Discover Weekly internals
- **Eugene Yan's Blog** — eugeneyan.com — recommender system deep dives
- **RecSys Conference Papers** — recsys.acm.org — industry papers (most are free)
- **FAISS (Facebook AI Similarity Search)** — github.com/facebookresearch/faiss
- **Annoy (Spotify's ANN library)** — github.com/spotify/annoy

Answers

A1. In the first week: the new song receives only an audio CNN embedding (from its spectrogram), because it has zero playlist co-occurrences and therefore cannot appear in word2vec training. After ∼30 days: the song has accumulated enough playlist appearances that word2vec will have learned a collaborative embedding for it. The serving pipeline then blends both the collaborative and audio-based embeddings. The transition is gradual, not sudden.

A2. The candidate generation model builds an ANN index over all items — rebuilding this index is expensive (it requires re-embedding every item in the catalogue). Hourly rebuilds would require massive GPU capacity and would frequently produce an unstable index. The ranker, by contrast, runs over only 1,000 candidates and has no index to rebuild; retraining hourly is cheap and captures within-day context shifts. If you retrained the candidate model hourly, the ANN index would be constantly stale during rebuild, causing recommendation gaps, and the cost would be prohibitive.

A3. To surface a new viral song within 24 hours, you would need to add a *trending signal fast path*: a real-time popularity counter (Kafka + streaming aggregation) that identifies songs with rapidly accelerating play counts. These trending songs would be injected directly into the candidate set without going through the weekly word2vec pipeline. The trade-off: you give up collaborative personalisation for those injected songs (everyone gets the same trending items), and you introduce a popularity bias that could crowd out personalised recommendations if the injection logic is too aggressive.

A4. **Two-tower advantage:** Separating user and item encoders allows you to pre-compute item embeddings offline and add new items without retraining the full model. It also scales to very large item catalogues. **Two-tower disadvantage:** The model cannot capture arbitrary user-item interactions in the embedding stage — the dot product interaction structure is relatively simple. Matrix factorisation, while even simpler, can be faster to train and easier to debug on small datasets, and lacks the representation capacity of deep towers.